



해아림

태양광 시간별 발전량 예측 B2B 서비스

서빙 → MLOps → AIOps 루프 위에서, 오차의 원인을 가려낸 뒤에만 재학습하는 운영 설계

● 실제 발전소 10 곳 · 3.7 년 실적 · Open-Meteo 기상

● 시나리오 9 개 · 재학습 2 회 · 새 클론에서 재현

예측제도는 " 맞힌 날 " 에만 돈을 준다

하루 전 10 시에 다음 날 24 시간을 제출하고 , 오차율이 작은 날만 정산금을 받는다

4% · 6%

정산 구간 . 2025 년 6 월부터 태양광 4% 이하 4 원 /kWh, 4~6% 3 원 /kWh. 그 전은 6%·8% 였다

D-1 10:00

제출 마감 . 이때 아는 것은 과거 실적과 기상 예보뿐

24h × N 곳

중개사업자 (VPP) 는 매일 발전소 수십 곳의 시간별 예측을 제출

운영자가 매일 부딪히는 네 가지

- 예측이 틀렸는데 원인을 모른다 . 날씨인지 , 설비 고장인지 , 모델이 낡은 것인지 .
- 오차가 커질 때마다 재학습하면 고장 난 발전소를 " 정상 " 으로 학습한다 .
- 점검이 필요한 발전소를 예측 오차만으로는 골라낼 수 없다 .
- 10 시 마감 전에 예측 API 가 멈추거나 느려지면 그날 제출을 못하고 , 제출이 없으면 정산도 없다 .

해아림이 답하는 것

- 내일 24 시간 발전량을 제도 포맷으로 API 한 번에 .
- 오차가 커지면 원인을 날씨 / 설비 / 모델로 자동 분류 .
- 모델 탓일 때만 재학습하고 , 게이트를 통과한 모델만 교체 .
- 설비 탓이면 재학습을 막고 점검 알림 .

제공 기능과 고객이 얻는 것

전력중개사업자 (VPP) 와 발전소 운영자를 위한 B2B API 와 운영 대시보드



다음 날 24 시간 예측 API

72 시간 실적과 24 시간 예보를 보내면 hourly_kwh[24] 와 model_version 을 돌려준다. 제출 자료 준비 시간이 사라진다.



실적 업로드와 사후 대조

컬럼 · 시각 · 중복 · 용량을 검증한 뒤 저장한다. 실적이 들어오면 날짜별 오차율과 PR 을 계산해 누적한다.



원인별 드리프트 대응

21 일 오차율이 임계값을 넘으면 PR · 동시성 · 모양으로 날씨 / 설비 / 오염 / 모델 드리프트를 가른다. 모델일 때만 재학습한다.



설비 점검 알림

그 발전소만 PR 이 0.75 아래면 [ALERT] 를 남기고 재학습을 막는다. 고장을 정상으로 학습하지 않는다.

HAIC 템플릿을 태양광 운영으로 바꾼 표

실습 가이드 7 번 도메인 매핑표 . 예측 대상부터 이해관계자까지 한 칸씩 치환했다

템플릿 개념 (HAIC)	실습 스켈레톤	해아림
예측 대상	다음날 증가 1 개	다음 날 24 시간 발전량 hourly_kwh[24] (제도 제출 포맷)
입력 시퀀스	최근 20 거래일 (close, volume)	최근 72 시간 실적 + 내일 24 시간 기상 예보 + 발전소 id
데이터 공급	대시보드 CSV 업로드	시간별 실적 CSV 업로드 (컬럼 · 시각 · 중복 · 용량 검증) + Open-Meteo 기상
배포 게이트	RMSE ≤ \$4.00	첫 배포 8% 통과율 ≥ 0.45, 이후는 현 운영 모델 이상 (같은 테스트 1 년)
드리프트 신호	변동성 3 배 급변 → RMSE > \$4	21 일 오차율 > 10.36% 이고 , PR · 동시성 · 모양으로 날씨 / 설비 / 오염 / 모델을 가름
재학습	최근 21 거래일 fine-tuning	모델 드리프트일 때만 , 영향 발전소의 최근 30 일로 fine-tuning (마지막 6 일은 검증용)
알림	aiops.log [WARN]	[WARN] 감지 · [ALERT] 설비 점검 · [INFO] 재학습 · [OK] 승격 (대시보드 로그 패널)
이해관계자	투자 분석가 , 운영자	전력중개사업자 (VPP) 운영팀 , 발전소 운영자 , 모델 담당자

운영 목표 : 응답 시간 · 가용성 · 품질

AI 솔루션이 지켜야 할 수치 . Lazy/Eager 는 같은 PC 에서 두 번씩 재서 적었다



응답 시간

/predict 한 건 1 초 이내 . 측정 75 ms (모델 로드 뒤) . 발전소 50
곳을 순서대로 불러도 4 초 안쪽



가용성

재학습은 백그라운드에서 돌고 그동안 /predict 는 기존 버전으로 응답 .
게이트에 걸리면 기존 버전 유지



품질

테스트 1 년 일 오차율 8.29%, 8% 통과율 0.579. 첫 배포 게이트는
통과율 0.45, 이후는 현 운영 모델 이상



입력 검증

길이 · 범위가 틀린 요청은 모델에 달기 전에 422 로 거절 . 어느 필드가 왜
틀렸는지 loc·msg 로 돌려준다

Lazy vs Eager 측정 (mlflow 모드 , CPU, 2 회씩)

모드	서버 시작 → /health 200	시작 직후 model_loade d	첫 /predict	두 번째 /predict
lazy 1 회	3.66 s	false	1,892 ms	76 ms
lazy 2 회	3.44 s	false	1,917 ms	73 ms
eager 1 회	5.04 s	true	342 ms	78 ms
eager 2 회	5.25 s	true	367 ms	75 ms

- Lazy 는 1.8 초 먼저 뜨지만 첫 사용자가 1.9 초를 기다리고 , 모델 파일 문제가 첫 요청 때 드러난다 .
- Eager 는 뜨는 데 1.6 초 더 걸리는 대신 /health 의 model_loaded=true 가 " 받을 준비 됐다 " 는 뜻이 된다 .
- Eager 도 첫 요청이 0.35 초인 것은 TensorFlow 가 첫 호출 때 그래프를 만들기 때문 . 기동 시 예열 호출 한 번이면 없어진다 .

● 선택 : 컨테이너 · 운영은 **Eager (헬스체크가 model_loaded 까지 확인)** , 개발 · 테스트는 **Lazy**

재현 : `python scripts/measure_loading.py → logs/loading_measure.json`

실제 데이터로 만들었다

공개 발전 실적과 Open-Meteo 기상 . 합성 데이터가 아니라 발전소 10 곳의 3.7 년치 시간별 실적이다

10 곳

시간별 계측이 정상인 발전소
(전체 24 곳 중)

3.7 년

2023-01 ~ 2026-08, 321,336
행

7 개 위치

Open-Meteo 관측과 하루 전
예보

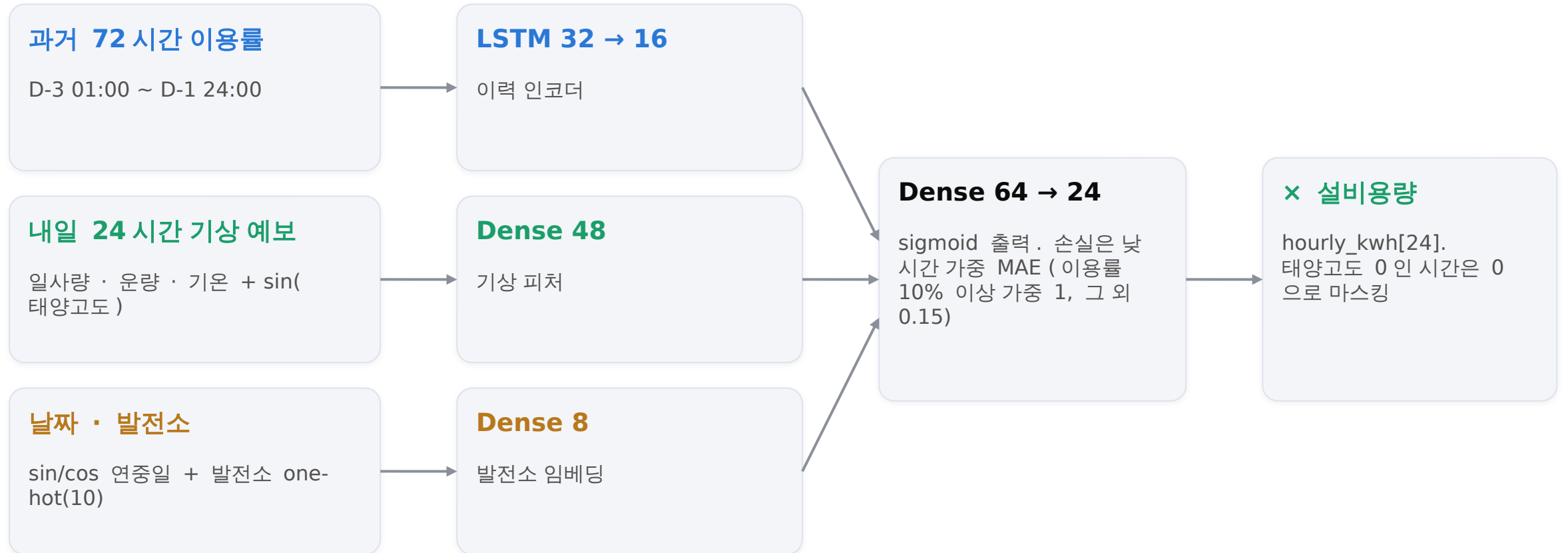
1 년

테스트 2025-09 ~ 2026-08,
3,534 발전소 - 일

항목	규칙	이유
예측 단위	이용률 = 발전량 ÷ 설비용량	용량이 다른 발전소 10 곳을 한 모델로 통합
시각 정렬	발전 "N 시 " = N-1~N 시 구간 , 기상은 N 시 직전 1 시간 평균	발전과 기상의 피크가 어긋나지 않게
제외 7 곳	시간별 급변 25~52%, 피크 3~4 시간 지연 , 장기 정지	계측 불량을 모델에 넣지 않는다
설비용량	시간 피크로 추정 (공식값은 plants.csv 에 넣으면 반영)	오차율의 분모이므로 공식값 확인이 필요
학습 / 평가	학습은 관측 기상 , 평가와 서빙은 하루 전 예보도 함께	서빙 조건의 오차를 따로 잰다

시간별 통합 모델 구조

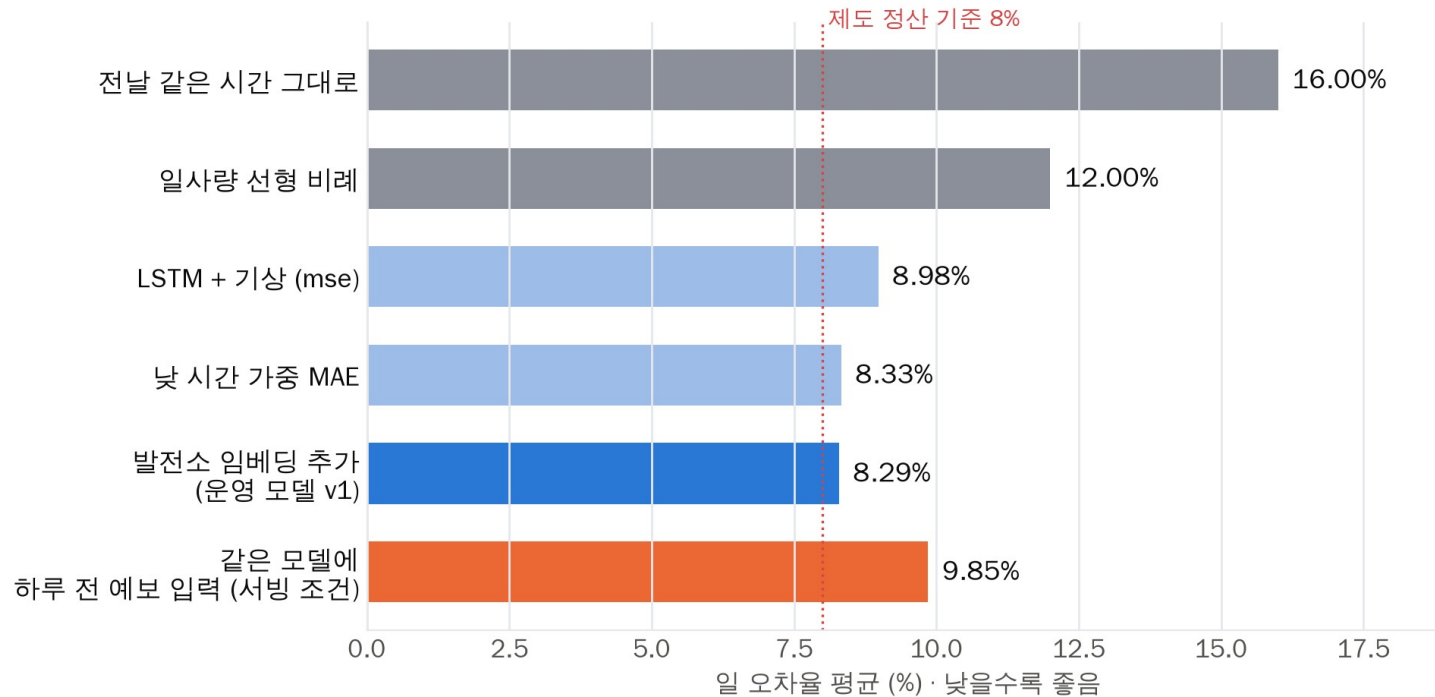
LSTM 이력 인코더 + 내일 기상·태양고도 + 날짜·발전소 임베딩 → 24 시간 이용률 . 파라미터 18,592 개



왜 이 구조인가 · mse 8.98% → 낮 가중 MAE 8.33% → 발전소 임베딩 7.99% (실험). 학습 길이 · 피쳐 추가 · 앙상블은 효과 없음 . 제도 오차율은 낮만 세므로 손실도 낮에 집중 .

기준선 대비 오차 절반

테스트 1년 · 발전소 10곳 · 3,534 발전소-일 · 일 오차율 = 용량 대비 시간 오차의 낮 시간 평균



8.29%

운영 모델 (관측 기상)

0.58

8% 통과율 (기준선 0.30)

9.85%

하루 전 예보 입력 (서빙)

0.49

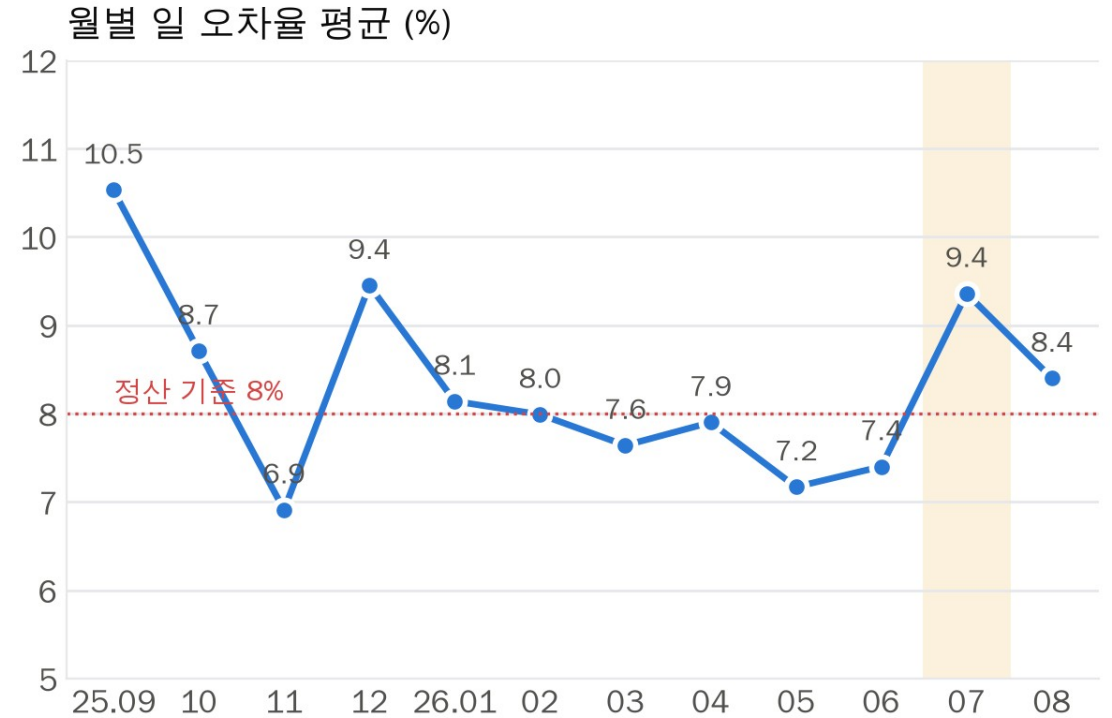
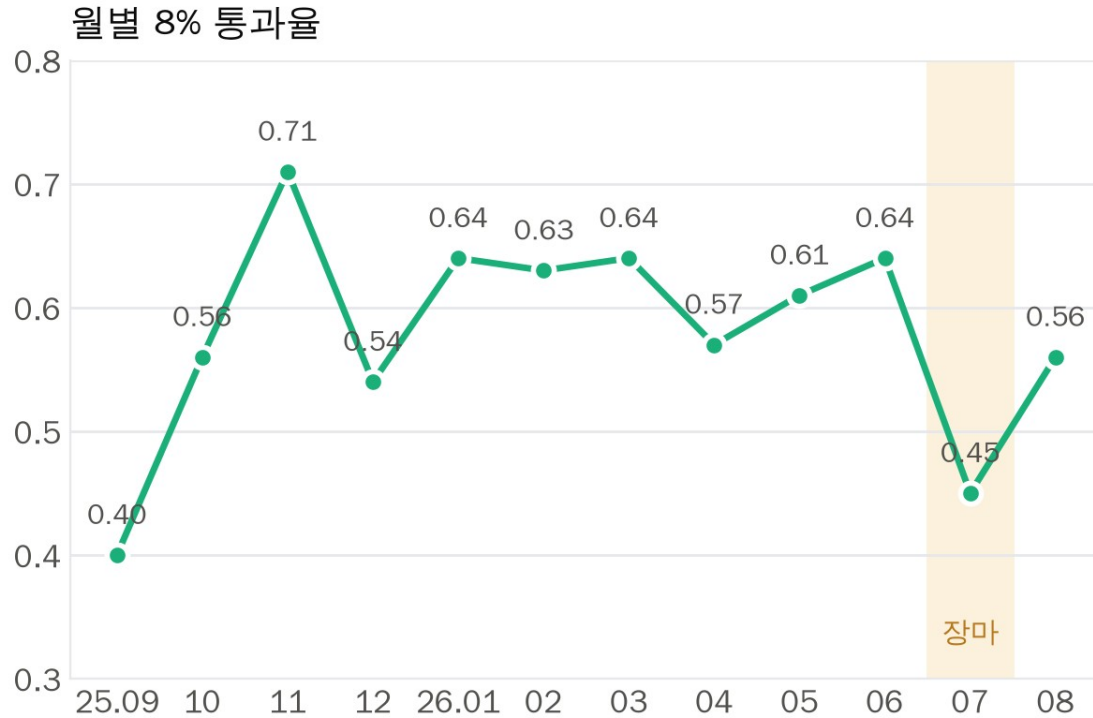
서빙 조건 통과율

예보 입력 격차 1.5%p

드리프트가 아니라 입력 품질의 몫이다. 예보 소스 개선, 최신 런 사용, 예보로 학습하는 것이 다음 과제. 재학습으로는 줄지 않는다.

계절 변동은 모델이 이미 아는 것

장마철 (7 월) 통과율 0.45 는 드리프트가 아니라 모델 성능의 한계다 . 가을 (11 월) 은 0.71



장마철 오차가 큰 이유 : 구름이 " 몇 시에 걷히느냐 " 를 시간 단위로 맞춰야 하고 , 평가에 남는 시간은 가장 변동이 큰 시간뿐이다 . 보수적 예측 손실과 예보로 학습이 다음 과제 .

지표 하나로 통일하고 , 게이트가 배포를 막는다

학습 손실 , 배포 게이트 , 드리프트 임계값이 모두 같은 제도 오차율에서 나온다

제도 오차율 (data/metrics.py)

시간 오차율 = $| \text{예측} - \text{실제} | \div \text{설비용량} \times 100$

발전량이 용량의 10% 이상인 시간만

일 오차율 = 시간 오차율의 평균 $\rightarrow$ 8% 이하를 내부 품질 기준으로
(제도 정산 구간은 4%·6% . 8%는 모델 통과율용 기준)

PR = $\sum \text{실제} \div \sum (\text{관측 기상으로 낸 기대치})$

1.0 = 설비가 모델이 아는 정상 상태

첫 배포

8% 통과율 ≥ 0.45

기준선 0.30 의 1.5 배 . 실제 0.579 로 통과

이후 배포

현 운영 모델 이상

같은 테스트 1 년에서 챔피언 / 챌린저 비교

재학습

남겨 둔 6 일에서 현 모델보다 오차가 낮을 때만

30 일 중 앞 24 일로 학습 , 뒤 6 일은 손대지 않고 두 모델을 같은 날짜로 비교 . 기후 B 에서는 4 곳 $\times$ 6 일 = 24 발전소 - 일 , 16.32% $\rightarrow$ 8.39%

드리프트 임계값은 검증 오차에서 유도한다

- 임계값 = $\max(8\%, \text{검증 오차 } 8.29\% \times 1.25) = 10.36\%$. 고정 8% 를 쓰면 평상시 (8.3%) 가 매일 드리프트가 된다 .
- 최근 21 일 일 오차율 평균이 임계값을 넘을 때만 원인 분류로 넘어간다 . 7 일 미만이면 보류 .
- 서버링 조건의 오차와 예보 괴리 (관측 - 예보 일사량 차이) 를 함께 기록해 " 예보가 빗나간 날 " 을 설명한다 .

드리프트는 모양으로 알아본다

성능비 PR 이 시간에 따라 그리는 모양. 같은 "오차 상승" 이라도 모양이 다르면 원인과 대응이 다르다



급락 (sudden)

하루 만에 계단형으로 떨어져 유지

예 : 삼천포 2 12/16

● 설비 이상 → **[ALERT]**

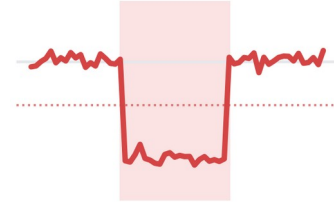


점진 (incremental)

몇 주 ~ 몇 달에 걸쳐 한 방향으로

예 : 영흥 #5 3~8 월

● 설비 · 오염 → 점검 권고

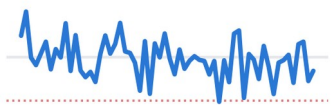


복귀 (recurring)

떨어졌다가 원래대로 돌아옴

예 : 광양항 11 월 , 삼천포 2 1/9

● 설비였다는 사후 확인

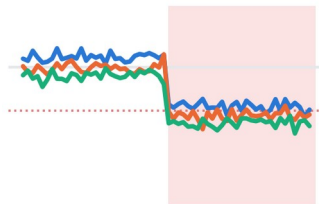


날씨 (no shift)

위아래로 흔들리지만 평균은 1.0

예 : 장마철

● 정상 → 재학습 없음

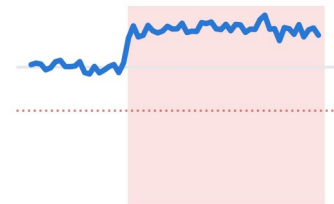


동시 변화 (concurrent)

여러 발전소가 같은 시기에 같은 방향

예 : 기후 B 폭염화 (모의)

● 모델 드리프트 → 재학습



위로 튕림 (PR > 1.15)

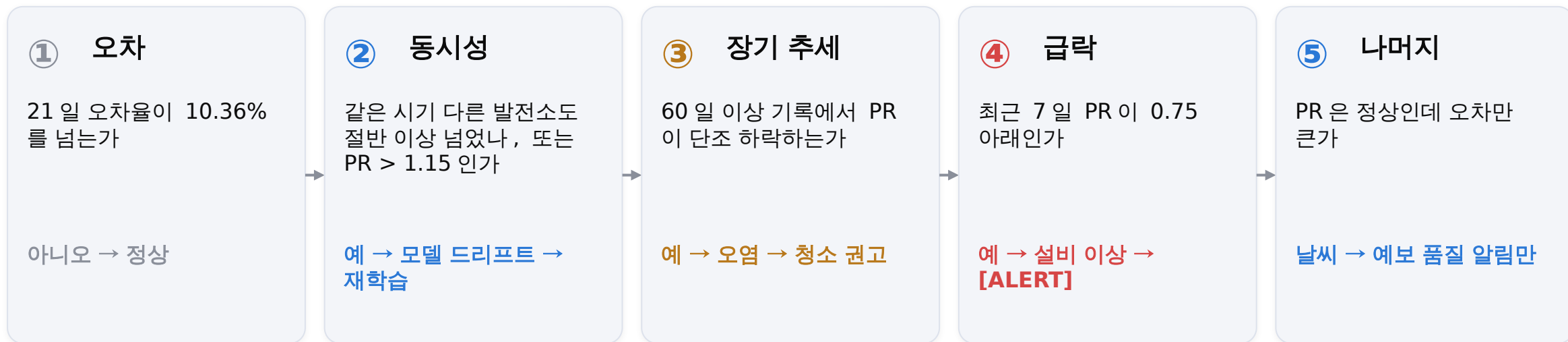
실제가 기대보다 계속 많음

예 : 광양항 복구 후

● 모델 기준 오류 → 재학습

오차가 커졌다는 것과 모델이 틀렸다는 것은 다르다

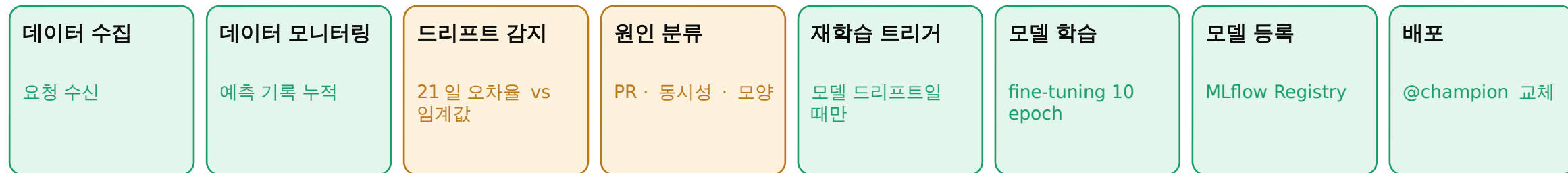
발전소가 기대보다 덜 만들 이유는 많지만 더 만들 이유는 없다. 다섯 단계로 원인을 가른다



원인	PR (실제 ÷ 기대치)	범위	모양	대응
날씨 · 예보 빗나감	≈ 1.0	한 지역	며칠	알림만
설비 고장 · 정지	0.75 아래로 급락	한 곳	계단형	[ALERT], 재학습 차단
오염 · 노후 · 그늘	0.75~0.92, 단조 하락	한 곳	몇 주 ~ 몇 달	청소 · 점검 권고
모델 드리프트	여러 곳 같은 방향, 또는 1.15 초과	절반 이상 동시	동시	fine-tune → 게이트 → 재배포

AIops 파이프라인 8 단계와 운영 안전장치

대시보드는 aiops.log 의 [WARN] → [INFO] → [OK] 를 읽어 단계를 켜다 . 모델 드리프트일 때만 5 단계로 넘어간다



재학습은 백그라운드 작업



/jobs/current 로 상태를 본다 . 학습 중에도 /predict 는 기존 버전으로 응답하고 , 진행 중이면 새 요청은 같은 작업으로 합친다 .

MLflow alias @champion



승격은 alias 이동이다 . 게이트에 걸리면 기존 버전을 유지하고 , 학습이 실패해도 서빙은 그대로다 .

재학습 데이터 선별



같은 시기에 PR 이 0.75 아래인 발전소는 동시성에 묶여도 제외한다 . 영흥 #5 를 섞으면 새 모델 오차가 10.41% 에서 10.95% 로 나빠졌다 .

검증과 테스트



업로드는 컬럼 · 시각 · 중복 · 미등록 발전소 · 용량 범위를 검사해 거부한다 . 계약 테스트 14 개 . 새 클론에서 전체 재현 .

시나리오 9 개 : 날씨 3 · 설비 4 · 모델 드리프트 2

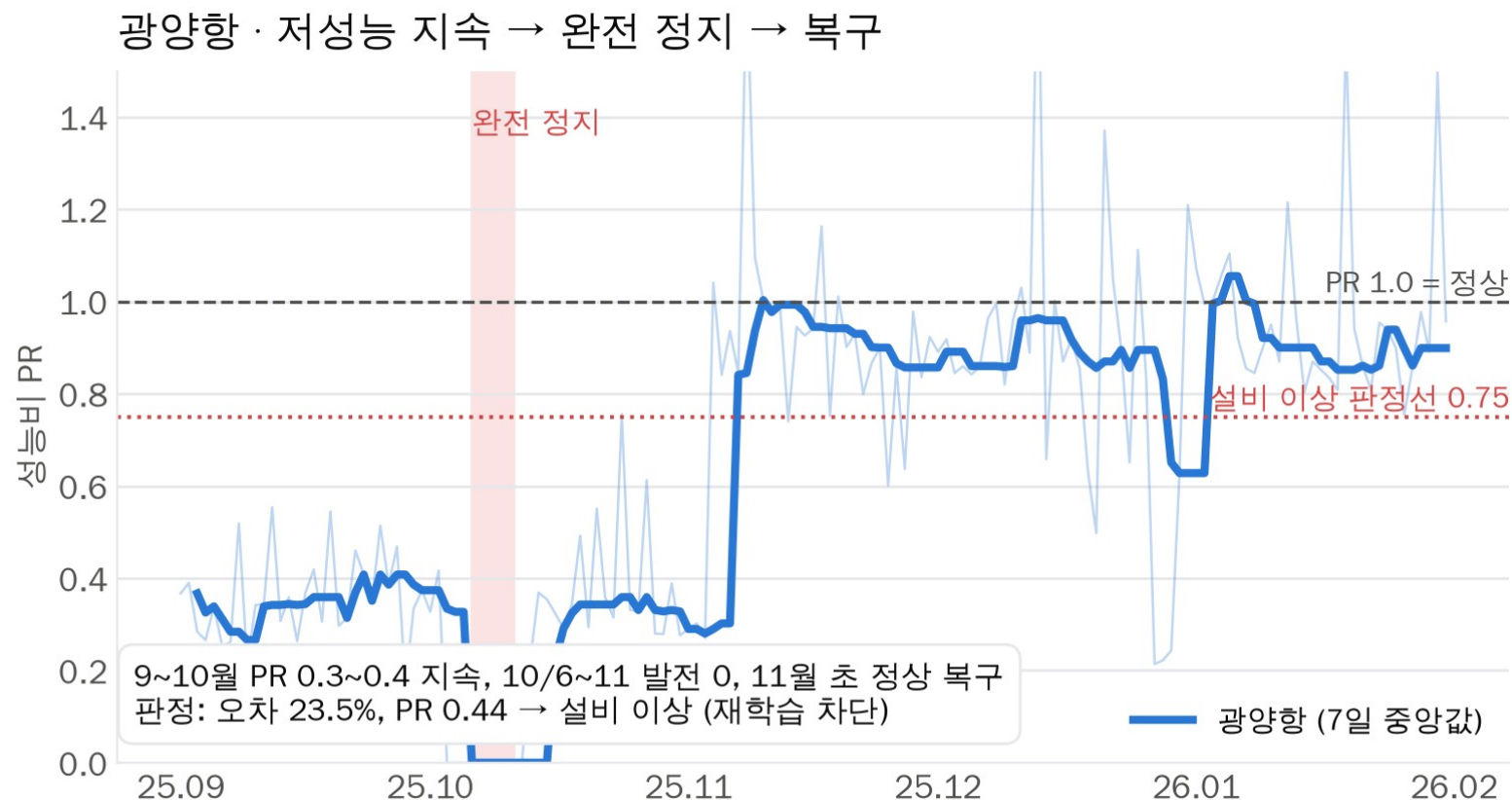
실제 실적 구간을 그대로 또는 변형해 투입했다. 재학습은 두 번만 걸렸고 둘 다 게이트를 통과했다

구분	시나리오	데이터	일 오차율	PR	판정 → 대응
날씨	정상	삼천포 2 2026-08-11~31	7.17%	0.98	정상
날씨	장마철	삼천포 2 2026-06-20~07-10	9.19%	1.05	정상 (모델이 아는 계절)
날씨	기후 A 우기 패턴만	6 곳 장마철 실적 그대로	6.9~9.2%	0.98~1.11	정상 (입력 분포만 변화)
설비	합성 고장	삼천포 2, 8 일째부터 $\times 0.5$	16.29%	0.46	설비 이상 → [ALERT]
설비	광양항 (실제)	2025-10-05~25, 10/6~11 정지	23.46%	0.44	설비 이상 → [ALERT]
설비	삼천포 2 (실제)	2025-12-05~25, 12/16 급락	15.81%	0.47	설비 이상 → [ALERT]
설비	영흥 #5 (실제)	2026-08-11~31, 3 월부터 점진 하락	11.69%	0.68	설비 이상 → [ALERT]
모델	기후 B 폭염화 (모의)	6 곳 2026-07-11~31, 28°C 초과 1°C 당 - 5%	10.9~15.0%	0.70~0.75	모델 드리프트 → fine-tune 8.39% → v2
모델	전 발전소 변화 (루프 검증)	경남 3 곳 $\times 1.25$	12.2~14.2%	1.19~1.22	모델 드리프트 → fine-tune → v3

● 같은 20% 손실을 장마 창에 넣으면 오차 8.5~9.2%로 임계값 미달. 제도 오차는 용량 대비 절대값이라 흐린 날의 비례 손실은 작게 잡힌다.

실제 사례 ① 광양항 : 정지했다가 돌아왔다

2025 년 9~10 월 PR 0.3~0.4 지속 , 10/6~11 완전 정지 , 11 월 초 복구 . 급락과 복귀 모양



판정

오차 23.5%, PR 0.44 → 설비 이상 .
[ALERT] 만 남기고 재학습 차단

근거

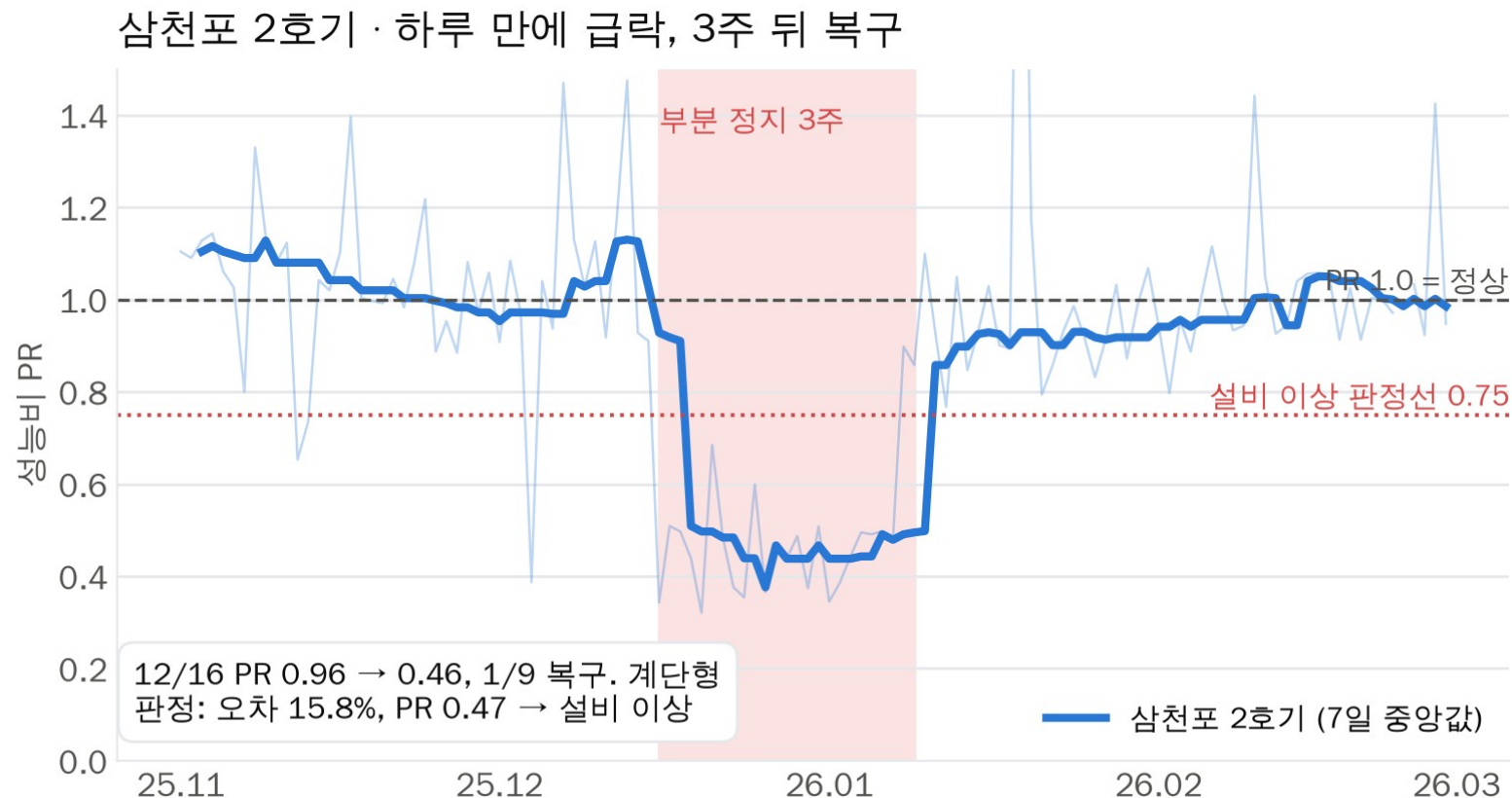
같은 기간 다른 9 곳 PR 0.85~1.10. 한
곳만 떨어졌다

배운 것

창 시작 전에 이미 떨어진 경우도 PR <
0.75 면 설비로 본다 (규칙 보강)

실제 사례 ② 삼천포 2 호기 : 하루 만에 절반

2025-12-16 PR 0.96 → 0.46, 3 주 유지 후 1/9 복구 . 전형적인 계단형 급락



판정

오차 15.8%, PR 0.47 → 설비 이상 .
[ALERT]

근거

같은 부지 삼천포 3 는 0.84 유지 . 하루
만에 떨어지고 하루 만에 돌아온 모양은 점
검 · 부분 정지

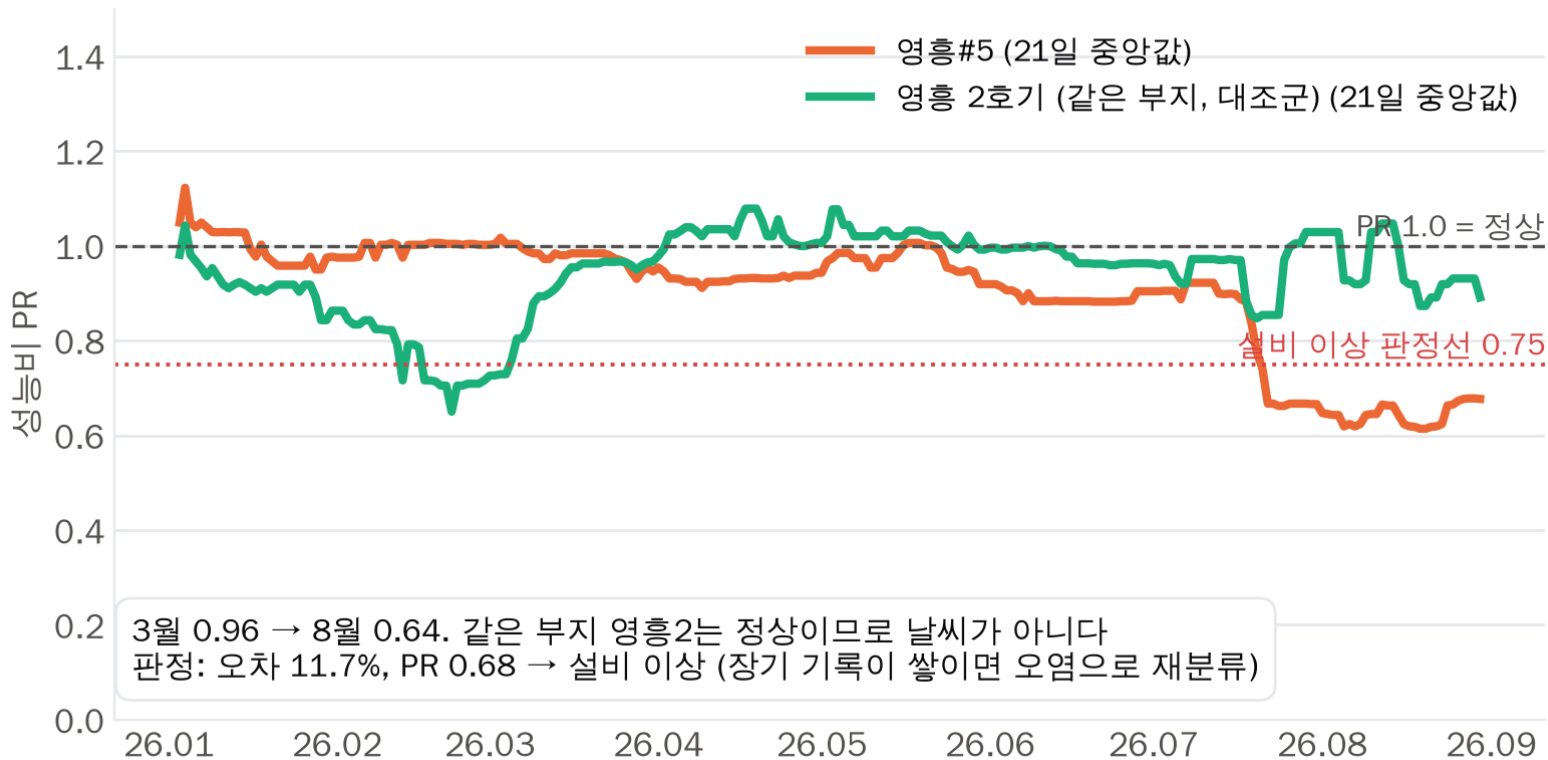
배운 것

합성 고장 시나리오 ($\times 0.5$) 와 같은 모양이고
감지기도 같은 판정을 냈다

실제 사례 ③ 영흥 #5: 여섯 달에 걸쳐 서서히

2026 년 3 월 0.96 → 8 월 0.64. 전 시간대 균일 손실 , 피크 이용률 0.84 → 0.78

영흥#5 · 6개월에 걸친 점진 하락



판정

오차 11.7%, PR 0.68 → 설비 이상 . 60 일 이상 기록이면 오염으로 재분류

근거

같은 부지 영흥 2 호기는 1.0 근처 . 날씨나 기상 데이터 문제가 아니다

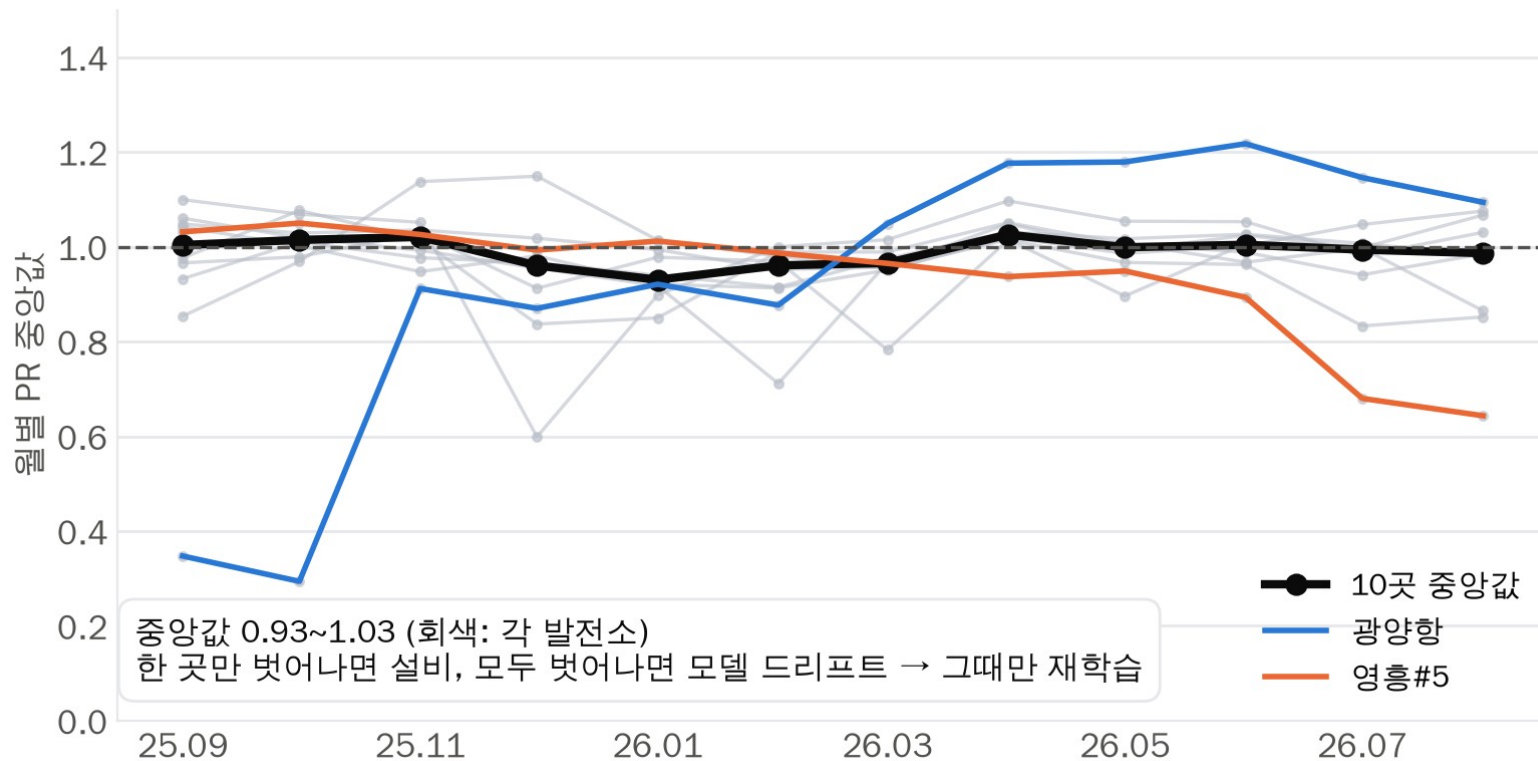
추정

스트링 · 인버터 일부 탈락 또는 심한 오염 . 운영 이력 대조 필요

지난 1년, 열 곳이 동시에 틀린 달은 없었다

월별 PR 중앙값 0.93~1.03. 재학습이 필요한 모델 드리프트는 실제 데이터에 없었다

발전소 10곳 월별 PR 중앙값 · 동시에 틀린 달이 없다



해석

이상은 전부 발전소 단위였고 감지기는 세 건 모두 재학습을 막았다

예외

광양항 복구 후 PR 1.15~1.22 (2026-04~07). 위로 틀린 유일한 구간

의미

재학습은 "데이터를 더 넣는 일"이 아니라
"잘못 배운 것을 바로잡는 일"

모델 드리프트 : 여러 곳이 같이 틀리거나, 위로 틀리거나

재학습이 정답인 경우는 " 같은 입력에서 발전 패턴이 바뀌었는데 메타데이터 수정으로는 못 고치는 " 경우다

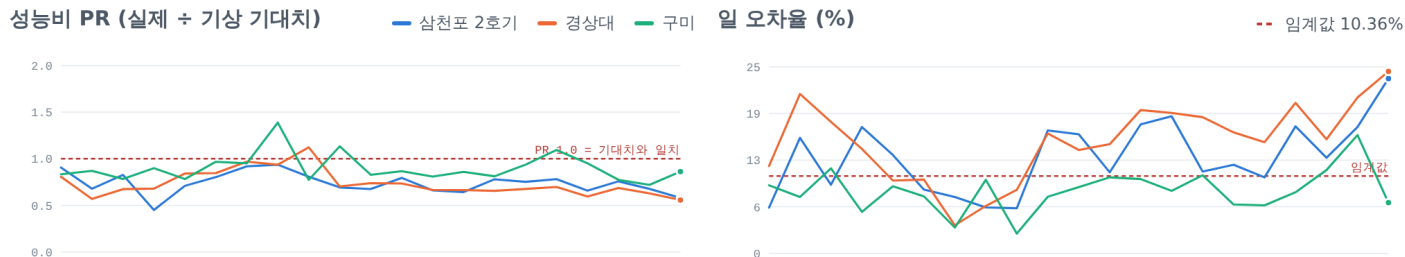
● model_drift · 4곳 동시 → 재학습 → v2 승격 (모의)

삼천포 2호기 2026-07-11 ~ 2026-07-31 · 경상대 2026-07-11 ~ 2026-07-31 · 구미 2026-07-11 ~ 2026-07-31

같은 6곳의 장마 직후 폭염 3주(7/11~31)에 관측 기온이 28°C를 넘는 시간마다 1°C당 5% 손실을 넣었습니다.
더운 경남 4곳은 오차 10.9~15.0%, PR 0.70~0.75로 임계값을 넘고 덜 더운 경북 2곳은 8.4~8.6%로 이내입니다.
다. 동시성 0.67로 모델 드리프트 판정, 4곳 최근 30일로 fine-tuning한 모델이 8.39%를 내 v2로 승격했습니다.
같은 손실을 장마 창에 넣으면 오차 8.5~9.2%로 임계값에 못 미칩니다. 제도 오차율은 용량 대비 절대 오차라 흐린 날의 비례 손실은 작게 잡히기 때문입니다.

발전소	일 오차율 평균	8% 통과율	PR 평균	임계값
삼천포 2호기	12.92%	0.19	0.737	10.36%
경상대	15.03%	0.095	0.735	10.36%
구미	8.41%	0.429	0.899	10.36%

성능비 PR (실제 ÷ 기상 기대치)



● 발전소는 덜 만들 이유는 많지만 더 만들 이유는 없다 → 위로 틀리면 모델을 의심한다

같은 20% 손실을 장마 창에 넣어도 임계값에 못 미친다 → 기후 시나리오는 맑은 폭염 창에 두었다

기후 B · 폭염화 (모의)

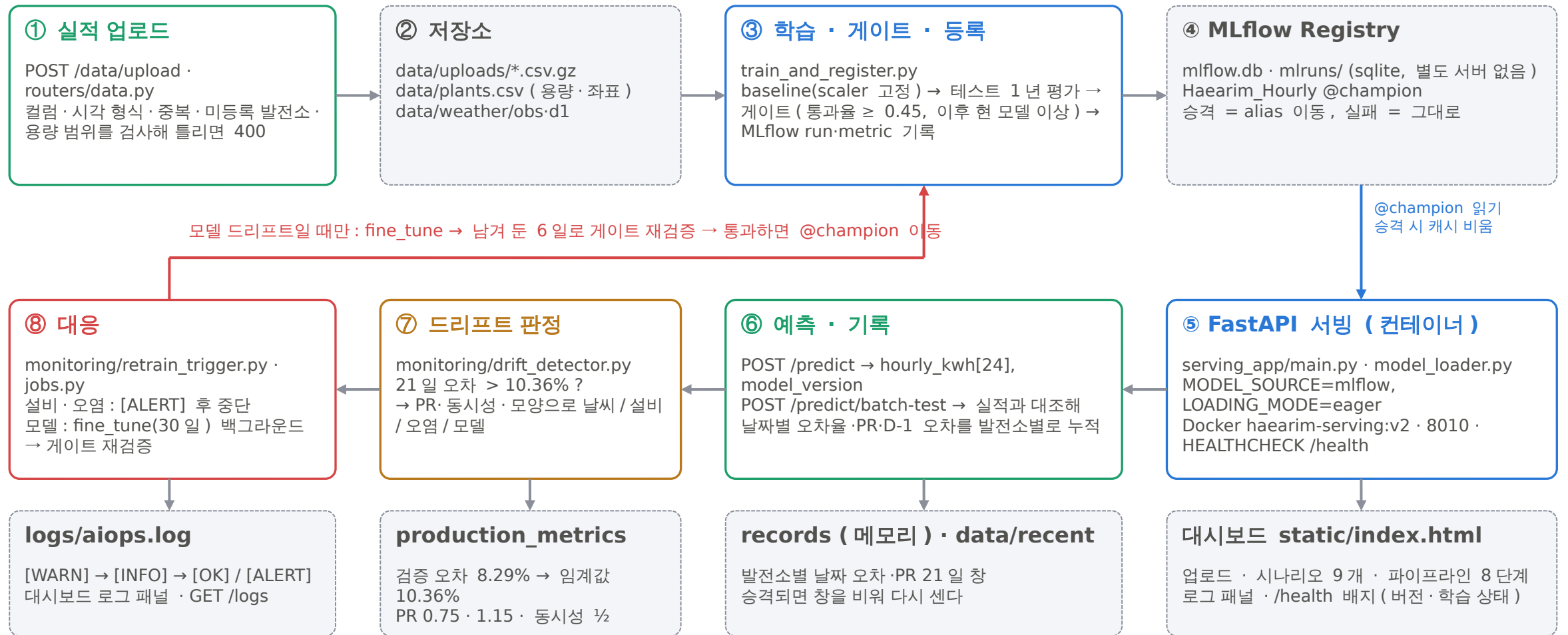
28°C 초과 1°C 당 - 5% 손실을 3 주 넣었다.
더운 경남 4 곳만 넘고 경북 2 곳은 이내 → 동시성 0.67 → 재학습 → 새 오차 8.39%로 v2 승격.
우기 패턴만 바뀐 A는 재학습하지 않는다.

광양항 복구 후 · 실제

넉 달 동안 실제가 기대보다 20% 많다. 구름·먼지·고장은 발전을 더 만들지 못하므로 틀린 쪽은 모델의 기준. PR > 1.15 규칙은 한 곳이어도 재학습으로 보낸다.

시스템 구성도 : 업로드에서 재배포까지 한 바퀴

위 줄은 만들어 내보내는 길 (Day1·Day2), 아래 줄은 지켜보고 고치는 길 (Day3). 모듈 이름은 저장소의 파일 이름 그대로다



실선 상자 = 코드 모듈 · 점선 상자 = 파일 · 저장소 · 화살표 = 호출과 읽기 · 쓰기 · 빨간 고리 = 모델 드리프트일 때만 도는 길

운영 API 계약

21

FastAPI 와 Pydantic 검증 . Swagger /docs 는 CDN 없이도 열린다 . 틀린 입력은 모델에 닿기 전에 422 로 돌려보낸다

Method / URL	입력	결과	용도
POST /predict	plant_id, date, history_kwh[72], forecast[24]{ghi, cloud_cover, temperature}	200 hourly_kwh[24], day_total_kwh, model_version · 422 길이 · 범위 위반	다음 날 제출 자료
POST /data/upload	CSV 또는 .gz (plant_id, time, generation_kwh)	200 rows, rejected_rows · 400 형식 · 중복 · 용량 위반	실적 갱신
GET /data/status · /data/window	plant_id, start, n_days, scale, heat_loss	발전소별 기간 / 시뮬레이션용 구간	운영 확인 · 시나리오
POST /predict/batch-test	records(72h+24h 이상), persist, check	days[], summary, drift_check{status, cause, retrain}	사후 평가 · 드리프트 판정
GET /jobs/current	-	idle / queued / running / completed / failed	재학습 상태
GET /health · GET /logs/aiops.log	-	status, model_loaded, model_version, loading_mode, training / 로그 본문	서비스 상태 · 증빙

200 정상 응답

```
POST /predict {"plant_id":"samcheonpo_2","date":"2026-08-30",
  "history_kwh":[0,0,...,559.68,...],"forecast":
[{"ghi":0,"cloud_cover":20,"temperature":23.1},...]}
→ 200 {"hourly_kwh":[0,0,0,0,0,0,0.8,107.6,211.1,334.4,432,519,534.4,...,0],
  "day_total_kwh":3649.7,"model_version":"production-v1"}
GET /health → {"status":"ok","model_loaded":true,"model_version":"production-
v1","loading_mode":"eager","training":"idle"}
```

422 검증 오류 (history_kwh 71 개를 보냈을 때)

```
POST /predict {... "history_kwh":[71개] ...}
→ 422 {"detail":[{"type":"too_short","loc":["body","history_kwh"],
  "msg":"List should have at least 72 items after validation, not 71"}]}
같은 식으로 ghi < 0, cloud_cover > 100, temperature 범위 밖, forecast 23개도
422.
업로드는 time 형식 · 중복 · 미등록 발전소 · 용량 초과를 400 으로 돌려보낸다 .
```

실행 증빙 ① 대시보드 : 설비는 멈추고, 모델 드리프트는 끝까지

버튼 한 번으로 실적 구간 투입 → 판정 → 파이프라인 8 단계 표시 (2026-10-01 실제 실행 캡처)



● 설비 고장 ×0.5 → 설비 이상 · 알림, 재학습 트리거 대기

오차 16.29%, PR 0.66. 파이프라인은 4 단계 (원인 분류) 에서 멈추고 재학습 트리거는 "대기 중". aiops.log 에는 [WARN] 과 [ALERT] 두 줄.

● 기후 B → 모델 드리프트 · 재학습 → 게이트 → v2 승격, 8 단계 완료

new_error 8.39% 로 새 버전 승격, 8 단계 모두 "완료". 상단 배지는 학습 중 "재학습 running" 으로 바뀌었다가 끝나면 모델 버전이 v2 로 바뀐다.

실행 증빙 ② 학습 중에도 응답하고, 로그는 순서대로 남는다

/health 가 재학습 running 을 보여주는 동안 /predict 는 production-v1 로 응답한다



해아림 v2 - 태양광 시간별 발전량 예측 Serving & AIOps 0.1.0 OAS 3.1

/openapi.json

default

POST	/predict	Predict	^
POST	/predict/batch-test	Batch Test	^
GET	/jobs/current	Current Job	^
GET	/predict/drift-state	Drift State	^
GET	/health	Health	^
POST	/data/upload	Upload	^
GET	/data/status	Status	^
GET	/data/window	Window	^
GET	/logs	List Logs	^
GET	/logs/{filename}	Read Log	^

```
17:00:50 [WARN] drift detected (cause=equipment, plant=samcheonpo_2, err=16.29% (thr 10.36%), pr_recent=0.464, pr_before=0.766, fleet_share=0.0)
17:00:50 [ALERT] equipment anomaly suspected at samcheonpo_2 (PR 0.464) - retrain blocked, operator notified
17:00:57 [WARN] drift detected (cause=equipment, plant=yeongheung5_1, err=11.69% (thr 10.36%), pr_recent=0.678, pr_before=0.653, fleet_share=0.5)
17:00:57 [ALERT] equipment anomaly suspected at yeongheung5_1 (PR 0.678) - retrain blocked, operator notified
17:01:24 [WARN] drift detected - triggering retrain (cause=model_drift, plant=samcheonpo_2, err=12.92% (thr 10.36%), pr_recent=0.712, pr_before=0.75, fleet_share=0.67)
17:01:24 [INFO] retrain triggered (window=last_30_days, plants=doosan_1,gyeongsang_1,samcheonpo_2,samcheonpo_3, job=345ed0baa11d)
17:01:41 [OK] new_error=8.39% - production promoted: Haeirim Hourly v2
17:01:56 [WARN] drift detected (cause=equipment, plant=samcheonpo_2, err=11.59% (thr 10.36%), pr_recent=0.574, pr_before=0.935, fleet_share=0.5)
17:01:56 [ALERT] equipment anomaly suspected at samcheonpo_2 (PR 0.574) - retrain blocked, operator notified
```

/predict 응답의 model_version

재학습 전 "production-v1"
 학습 중 /health {"training": "running"} · 응답은 여전히 v1
 승격 후 "production-v2"

실행 증빙 ③ 같은 요청 , 재학습 전과 후

Swagger /predict Try it out. 같은 삼천포 2 입력에 production-v1 과 production-v2 가 다르게 답한다

Server response	
Code	Details
200	Response body <pre>{ "plant_id": "samcheonpo_2", "date": "2026-08-30", "hourly_kwh": [0, 0, 0, 0, 0, 0, 0, 0.8, 107.6, 211.1, 334.4, 432, 519, 534.4, 461.1, 407.3, 335.6, 212, 94.3, 0.1, 0, 0, 0, 0, 0], "day_total_kwh": 3649.7, "model_version": "production-v1" }</pre> Response headers <pre>content-length: 234 content-type: application/json date: Fri, 02 Oct 2026 01:45:05 GMT server: uvicorn</pre>

Server response	
Code	Details
200	Response body <pre>{ "plant_id": "samcheonpo_2", "date": "2026-08-30", "hourly_kwh": [0, 0, 0, 0, 0, 0, 0, 1.1, 66.8, 159.7, 234.4, 305.4, 379, 402.7, 317.3, 286.8, 289.2, 178.8, 87.4, 0.1, 0, 0, 0, 0, 0], "day_total_kwh": 2708.7, "model_version": "production-v2" }</pre> Response headers <pre>content-length: 233 content-type: application/json date: Fri, 02 Oct 2026 01:45:23 GMT server: uvicorn</pre>

● 재학습 전 · "model_version": "production-v1" · 3,649.7 kWh

● 재학습 후 · "model_version": "production-v2" · 2,708.7 kWh

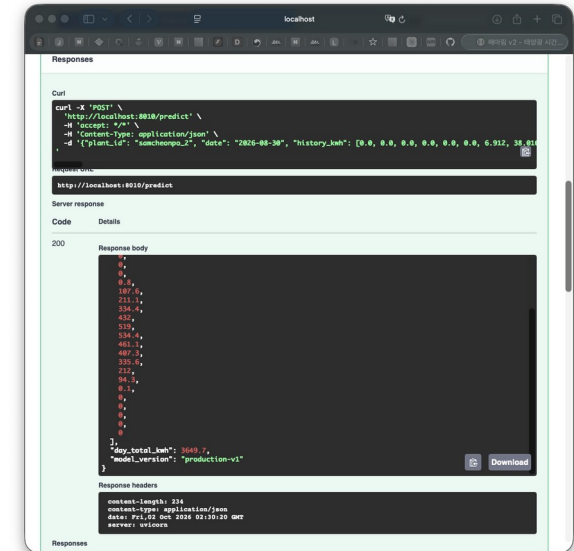
폭염 손실을 배운 v2 가 같은 날 발전량을 낮게 본다 . 승격 직후 /health 는 model_loaded=false(캐시 비움) 였다가 첫 요청에서 v2 를 올린다 .

실행 증빙 ④ 컨테이너에서도 같은 화면

docker compose up --build 한 번으로 시드 → 등록 → 테스트 → 서빙까지 이미지 안에서 끝난다 (Mac, 2026-10-02)

```
project_solar -- pkil -f "uvicorn serving_app.main:app", docker compose -f up --build --pkil -- docker-compose -f doc...
=> [2/9] WORKDIR /app 0.2s
=> [3/9] COPY requirements.txt . 0.0s
=> [4/9] RUN pip install --no-cache-dir -r requirements.txt 33.4s
=> [5/9] COPY serving_app/ serving_app/ 0.2s
=> [6/9] COPY data/ data/ 0.0s
=> [7/9] COPY scripts/ scripts/ 0.0s
=> [8/9] COPY tests/ tests/ 0.0s
=> [9/9] RUN m -r f data/uploads data/recent mlruns mlflow.db logs && mkdir -p data/uploads logs && cp data/sample 7.79s
=> exporting to image 37.9s
=> exporting layers 30.8s
=> exporting manifest sha256:1dcf4c4e1a6fde5734f095075130802309f8b703466aa946772808431 0.0s
=> exporting config sha256:8d8dbff164ea7b35a3b1f4472e4390211f1a105e2d0dc8324789517de8534 0.0s
=> exporting attestation manifest sha256:1a80b70e2b06deada50f7919f28b0decefe8ed196c58dcd76770e5312245 0.0s
=> exporting manifest list sha256:08092312d0c38007780b03040e014493c201cc3fdd0b0c4e43f872942 0.0s
=> naming to docker.io/library/haearim-serving:v2 0.0s
=> unpacking to docker.io/library/haearim-serving:v2 7.7s
=> resolving provenance for metadata file 0.0s
[+] up 4/4
Image haearim-serving:v2 Built 87.6s
Network serving_app_default Created 0.3s
Volume serving_app_haearim-logs Created 0.1s
Container serving_app-serving-v2 Created 0.5s
Attaching to serving-app-1
serving-app-1 | INFO: Started server process [1]
serving-app-1 | INFO: Waiting for application startup.
serving-app-1 | [eager] model loaded in 0.576s at startup
serving-app-1 | INFO: Application startup complete.
serving-app-1 | INFO: Uvicorn running on http://0.0.0.0:8010 (Press CTRL+C to quit)
serving-app-1 | INFO: 192.168.65.1:28491 - "GET /docs HTTP/1.1" 200 OK
serving-app-1 | INFO: 192.168.65.1:28491 - "GET /swagger-static/swagger-ui.css HTTP/1.1" 200 OK
serving-app-1 | INFO: 192.168.65.1:44482 - "GET /swagger-static/swagger-ui-bundle.js HTTP/1.1" 200 OK
serving-app-1 | INFO: 192.168.65.1:28491 - "GET /openapi.json HTTP/1.1" 200 OK
serving-app-1 | INFO: 192.168.65.1:44482 - "GET /health HTTP/1.1" 200 OK
serving-app-1 | INFO: 192.168.65.1:28491 - "GET /swagger-static/favicon-32x32.png HTTP/1.1" 200 OK
serving-app-1 | INFO: 127.0.0.1:16406 - "GET /health HTTP/1.1" 200 OK
serving-app-1 | INFO: 192.168.65.1:63489 - "GET /health HTTP/1.1" 200 OK
serving-app-1 | INFO: 192.168.65.1:63489 - "GET /health HTTP/1.1" 200 OK
serving-app-1 | INFO: 192.168.65.1:63489 - "GET /health HTTP/1.1" 200 OK
serving-app-1 | INFO: 127.0.0.1:145680 - "GET /health HTTP/1.1" 200 OK
serving-app-1 | INFO: 192.168.65.1:63489 - "GET /health HTTP/1.1" 200 OK
serving-app-1 | INFO: 192.168.65.1:63489 - "GET /health HTTP/1.1" 200 OK
```

```
project_solar -- iminglul-MacBookPro -- zsh -- 120x44
docker-compose -f serving_app/docker-compose.yml up --bu
Last login: Thu Oct 1 15:01:39 on ttys000
project_solar git:(main) docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
ff9c35172918   haearim-serving:v2                 "uvicorn serving_app." 59 seconds ago Up 58 seconds (healthy)   0.0.0.0:8010->8010/tcp, [::]:8010->8010/tcp
project_solar git:(main)
```



docker compose -f serving_app/docker-compose.yml up --build

이미지 빌드 87.6 초 → [eager] model loaded in 0.576s → Uvicorn running :8010. 아래 GET /health 줄이 HEALTHCHECK

docker ps

Up 58 seconds (healthy). HEALTHCHECK 가 /health 의 model_loaded=true 까지 확인한 뒤에야 healthy

http://localhost:8010/docs (컨테이너)

tests/sample_predict_request.json 을 보내 200. 로컬과 같은 3,649.7 kWh, production-v1

Dockerfile 줄	하는 일
FROM python:3.11-slim · COPY requirements.txt · RUN pip install	로컬과 같은 Python, 의존성 레이어는 소스보다 먼저 (캐시)
RUN cp sample → data/uploads && train_and_register.py --register-local && pytest	이미지 빌드 시점에 시드 · 게이트 · 등록 · 테스트까지 끝내 자기 완결
ENV MODEL_SOURCE=mlflow LOADING_MODE=eager · HEALTHCHECK · CMD uvicorn :8010	뜨자마자 모델이 올라와 있고 , healthy 가 곧 " 받을 준비 됨 "

한계와 다음 단계

모의임을 밝히고, 운영에 넣기 전에 확인할 것을 적는다



이력 창 마감 시각

지금은 D-1 24:00 까지 쓴다. 제출이 D-1 10 시이므로 09:00 마감으로 바꾸면 조건이 정직해지고 오차는 조금 오른다.



감시 오차의 기준

판정은 관측 기상 예측 오차 (8.29%) 기준이다. 서빙 조건 (9.85%) 으로 옮기면 임계값도 다시 잡아야 한다.



설비용량·좌표

plants.csv 의 용량은 시간 피크 추정값이다. 공식 용량·좌표와 세 발전소의 운영 이력을 대조해야 사례가 확정된다.



장마철 통과율 0.45

드리프트가 아니라 모델 성능의 한계다. 보수적 예측 손실과 예보로 학습하는 것이 다음 과제다.



시나리오 확장

출력제어 (제도 변화 → 재학습) 와 황사 (지역 동시지만 일시적 → 재학습 금지) 로 동시성 규칙의 경계를 보강한다.

역할 분담과 실행 안내

저장소 github.com/leemingi0906/model-serving · 팀 공유 zip [Haearim_v2_team.zip](#) · 3~5 분이면 같은 화면이 뜬다

담당	구현한 것	완료 증거
A 데이터 · 기획	시각 정렬, 이용률, 레지스트리, 업로드 검증, 기획안	정합성 검토, 7 곳 제외 근거, 계약 테스트
B 모델 · MLOps	LSTM 구조, 낮 가중 손실, 게이트, MLflow alias	게이트 통과 8.29% / 0.579, 모델 실험 2 라운드
C 서빙 · 배포	스키마, /predict, 모델 스냅샷 교체, 오프라인 Swagger	24 시간 응답, model_version v1 → v2, 학습 중 응답
D AIOps · 검증	원인 분류, 재학습 트리거, 백그라운드 작업, 시나리오 9 개, 대시보드	aiops.log, 새 클론 재현 로그

팀원 PC 에서 (TEAM_GUIDE.md)

```

pip install -r requirements.txt
bash scripts/quick_start.sh      # 업로드 → 동봉 모델 등록 → 서버
python scripts/simulate_drift.py # 시나리오 9 개
python -m pytest -q              # 계약 테스트 14 개
python scripts/measure_loading.py # Lazy/Eager 측정표
docker compose -f serving_app/docker-compose.yml up --build
http://localhost:8010 · /docs

```

발표의 한 줄

"오차가 커져도 날씨면 둔다, 한 곳이면 고친다, 모두면 다시 배운다."

재학습은 데이터를 더 넣는 일이 아니라 모델이 잘못 알고 있는 것을 바로잡는 일이다. 그래서 원인을 먼저 가린다.